The Ghost in the Machine

Precog NLP Recruitment Task Report

Task Completion Summary

Tasks Completed in Full

- **Task 0:** Dataset construction for all three classes (Human, Generic AI, Mimic AI), text segmentation, train/val/test stratified splitting.
- **Task 1:** Exploratory Data Analysis: 15 linguistic features engineered and visualised, PCA and UMAP dimensionality reduction, statistical significance testing, feature importance ranking.
- **Task 2:** Three classifiers trained and evaluated: Tier A (XGBoost), Tier B (Feed Forward Neural Network with averaged GloVe embeddings), and Tier C (DistilBERT + LoRA). All three evaluated on the held-out test set with accuracy, macro F1 score and confusion matrices.
- **Task 3:** Integrated Gradients saliency analysis, near-miss analysis, adversarial testing
- **Task 4:** Genetic Algorithm (7 generations) to evolve AI-generated text towards P(Human) ≥ 0.90 against the Tier C classifier. Personal essay classification test with a comparison against an AI-style rewrite.

Modifications and Deviations

I did not skip any part. However, I had to make two modifications.

Task 3:

Since my model achieved 100% accuracy on the test set, there were no misclassification examples to analyse. The task originally asked me to find three samples where Human was labelled as AI and run error analysis on them. I could not do that because there were no such samples. So I adapted the methodology to still extract meaningful insights about how the model makes its decisions.

Instead of error analysis, I ran **Integrated Gradients** saliency analysis on correctly classified samples to see which tokens the model actually relied on per class, performed a **near-miss analysis** on the least confident correct predictions (the most AI like human paragraphs and the most human-like AI paragraphs), and constructed targeted **adversarial paragraphs** to test the decision boundary. I believe this gave me more insight than three random misclassifications than three random misclassifications would have.

Task 4:

The Genetic Algorithm was set up with a target of P(Human) ≥ 0.90. The GA ran for 7 generations peaked at P(Human) = 0.4945, and plateaued from generation 5 onwards. The 0.90 target was never reached. I did not treat this as a failure.

Task 3's adversarial probe had already shown that the model requires multiple features to align Simultaneously: past tense, Victorian vocabulary, and syntactic complexity. Attacking a single feature at a time was not enough to flip the classification and GA confirmed this. The plateau at around 0.49 is the finding, not a shortcoming.

Task 0: The Library of Babel

The aim was to construct a balanced three-class dataset of 1,500 text paragraphs: 500 from real Victorian novels (Class 1 - Human), 500 generated by Gemini in neutral modern prose (Class 2 - Generic AI), and 500 generated by Gemini prompted to mimic the author from Class 1 (Class 3 - Mimic AI).
This dataset serves as the base for all downstream classification tasks.

0.1 Author Selection and Source Texts

First and foremost, I had to choose authors whose writing styles were distinct enough from each other, so that classification signals would reflect style instead of topic overlap.

Mary Shelley: *Frankenstein* (Project Gutenberg #84)

While looking for summaries on the internet, I saw that readers describe Shelley's writing style in *Frankenstein* as "long epistolary philosophical with a Gothic tone." Her sentences have complex nested clauses and a very formal Victorian vocabulary. This novel has the Shelley's style as the anchor for Class 1 and the explicit style target for Class 3 (Mimic AI). [1] [2]

Charles Dickens: *Great Expectations* (Project Gutenberg #1400)

Dickens used shorter sentences than Shelley and a direct, conversational tone. He used a technique called "onomatopoeia". His character names hint at personality. I chose these two very different authors because the contrast between them means that any classification signal that my models would develop will reflect the authors' style. [3]

Both authors belong to the *Victorian era (1837–1901*) and write on similar themes: *society*, *morality*, and *human nature*. My sister helped me choose these two for their stylistic differences.

Mary Shelley: *Mathilda* (Project Gutenberg #15238)

After performing initial segmentation of *Frankenstein*, I only got 376 filtered paragraphs. This was short of my 500-per-author target. I could have relaxed the word-count filter but that would have introduced length inconsistency across classes. Hence, I chose a second Shelley novel, Mathilda. I chose it because it shares the same author voice as Frankenstein, has sufficient length to give me the needed paragraphs after filtering, and has overlapping themes: *gothic atmosphere*, *isolation*, *rejection*.

I pooled the two Shelley texts and randomly sampled 500 segments from the combined 556-paragraph pool. Great Expectations already gave me more than 500 paragraphs, so I selected the first 500.

0.2 Topic Extraction

I manually extracted ten broad themes from the three novels by skimming the opening and closing chapters and cross-referencing online theme summaries. I will use these topics to guide Gemini's paragraph generation for Class 2 and Class 3. [4] [5] [6] [7] [8] [9] [10]

Mary Shelley (Frankenstein and Mathilda):

- The hubris of scientific creation
- The isolation and rejection of the outcast
- The ethics of love, parenthood, and abandonment
- Grief and the impossibility of recovery
- Nature and the sublime as emotional mirror

Charles Dickens (Great Expectations):

- Social class and the illusion of gentility
- Wealth, ambition, and moral corruption
- Crime, guilt, and redemption
- Self-deception and identity
- The past as a trap

Note: A few themes overlap loosely across both the authors (*isolation*, *guilt*, *the weight of the past*). I did this on purpose so that the AI-generated paragraphs for Classes 2 and 3 will have a broad topic coverage. Thus, any classification signal will more cleanly reflect style rather than topic.

0.3 Text Cleaning

I implemented a cleaning pipeline and passed each novel through it before segmentation:

- Normalise line endings from *\r\n* to *\n*.
- Remove chapter and volume headings using regex (e.g., "CHAPTER IV").
- Remove illustration placeholders (*[Illustration]*).
- Strip trailing whitespaces from each line.
- Collapse consecutive blank lines to a maximum of two.

I manually removed Project Gutenberg headers and footers since they are consistent across texts and a manual pass was faster and safer than writing edge-case regex. I also removed the table of contents in Great Expectations manually.
Mathilda had non-standard elements: a preface by the editor, introductory remarks, a contents list, and inline footnotes. I had to manually pre-clean Mathilda and then passed it through the same pipeline as the other texts.

0.4 Text Segmentation

The goal was to split cleaned texts into segments (paragraphs) of 150–200 words with sentence-coherent boundaries. I chose this paragraph lengthy just to match the target length of the AI-generated paragraphs in Classes 2 and 3.

First Attempt: Regex-based sliding window:

My script accumulated words until the count reached more than or equal to 150, then added more until the count hit 200 or a regex-detected sentence boundary appeared. This approach failed because my regex patterns detected only punctuation position and ignore actual structure. I observed that abbreviations like "Dr." and "Mrs." were incorrectly treated as sentence endings, hence causing segments to begin or end mid-clause. The mean word count came out to be 199 words because I was forcing the window to stop exactly at the punctuation boundary. [11]

Second Attempt: SpaCy NLP-based Segmentation:

I used SpaCy's `***doc.sent***` to identify true sentence boundaries. Then I grouped complete sentences within the 150–200 word range. This way, I got a more natural distribution with a mean of around 185 words per paragraph across all three texts. [12] [13]

Segment counts after filtering: Frankenstein - 396, Great Expectations - 945, Mathilda - 160.

After pooling and sampling, the final Class 1 dataset contains 250 Shelley segments and 250 Dickens segments (500 total), hence balanced for downstream training.

0.5 AI Dataset Generation (Classes 2 and 3)

I used the Gemini API to generate 50 paragraphs per topic for each class, with the following configuration choices:

- **Model:** *gemini-3.1-flash-lite-preview* because it offered the highest free-tier requests-per-day (RPD) limit among available Gemini models. (checked on March 19th, 2026)
- **Temperature:** Temperature 0.9 for Class 2 (Generic AI) because I wanted to maximise lexical diversity across the neutral paragraphs. Temperature 0.7 for Class 3 (Mimic AI) to focus Gemini more on the instructed Victorian style.
- **Prompts:** I wrote the Class 2 prompt to instruct Gemini to write in "neutral, clear, modern English prose" without imitating any specific author. Class 3 prompt instructed it to write in the style of Mary Shelley: long complex sentences with nested clauses, formal Victorian vocabulary, first-person philosophical tone, and Gothic register.

- **Checkpointing:** After each successful API call, my code appended valid paragraphs (130–200 words) to a topic-level checkpoint file. Hence, a mid-run API failure could be resumed instead of being restarted from scratch.
- **API Key Rotation:** I got four API keys from different accounts, loaded and managed them using a *KeyRotator* class that automatically switched to the next key on any *429/quota/rate-limit* error. I used a 5-second inter-call sleep to stay under the 15 RPM limit of the model.

0.5.1 Observation, Analysis and Solution

Class 2 generation completed with 480 paragraphs (20 short of 500) because *gemini-3.1-flash-lite* returned 503 "server unavailable" errors on the 10th topic. I knew this would be a risk of using a preview model but the 500 RPD limit was too lucrative. I wrote a top-up script to identify shortfall topics from checkpoint data and filled them using *gemini-2.5-flash* as a fallback.
After running the script, I saw a logic bug that the checkpoint records were appended along with newly generated records. This produced 530 entries for Class 2. I corrected it by de-duplicating on the *text* field in my checkpoints and trimmed them to 50 records per topic.

Class 3 generation was a nightmare: only 275 of 500 paragraphs were generated in the initial run because Gemini API returned constant server-side errors. I generated the remaining 225 paragraphs in a second top-up pass using *gemini-2.5-flash* because I could not risk hitting the server-side errors again with the *gemini-3.1-flash-lite* model.
This means approximately 45% of the Class 3 dataset came from a different model than the remaining 55%. I have disclosed this mixed-model composition as a data limitation in my code notebook.
I observed that *gemini-2.5-flash* failed to respect the word-count constraint in the prompt several times. My word-count filter logic caught and discarded it but I am glad my output validation worked well.

0.6 Final Dataset and Train/Val/Test Split

The final combined dataset contained 1,500 paragraphs: 500 per class. I down-sampled Class 1 from 1,000 (500 Shelley + 500 Dickens) to 500 (250 per author) to maintain class balance with the other two classes. Without this down-sampling, my future models would see twice as many Human examples during training. This would risk predictions being biased toward the Human class.

I split the dataset into **train**/**val**/**test** in a 70/15/15 ratio using stratified sampling (*sklearn.model_selection.train_test_split* with *stratify=y*). I used stratification to ensure that each split maintained exactly one-third of each class.

Final split sizes: Train - 1,049 records, Val - 226 records, Test - 225 records.

0.7 AI and Tool Usage

I attempted topic extraction with ChatGPT and Gemini but both returned surface-level textbook summaries that were too generic to use as generation prompts. I extracted the final topics manually by skimming the novels and cross-referencing online theme analyses (cited).

Github Copilot in my VS Code IDE (free-tier) structured the API key rotation class and the checkpointing logic. The idea for detecting rate limits was inspired by the documentation, which I navigated. I modified Copilot's output at almost every step because free-tier Copilot cannot see the full database as context and often gave suggestions that conflicted with my data structure. I wrote the generation prompts, the validation logic, the data cleaning pipeline, etc., with Copilot assisting me with boilerplate codes.

Task 0 References (hyperlinked)

1. Frankestein - Epistolary form of writing. Rajuel's Blog.
2. What is Mary Shelley's writing style in Frankenstein? Enotes.
3. Use of language in Great Expectations. BBC Bitesize.
4. Frankenstein Themes. LitCharts.
5. Frankenstein Themes. SparkNotes.
6. Common interpretation of Frankenstein. Reddit.
7. Great Expectations Themes. SparkNotes.
8. Great Expectations Themes. LitCharts.
9. Great Expectations Themes. Enotes.
10. Mathilda Themes. SparkNotes.
11. Regular Expressions, Text Normalization, and Edit Distance. Jurafsky's NLP Lecture Notes.
12. Linguistic Features. SpaCy Documentation.
13. SpaCy Doc.sents. TutorialsPoint.

Task 1 — The Fingerprint

This is the EDA phase of the assignment. The goal was to engineer a set of stylometric features from the three-class dataset, visualise their discriminative power, and prepare a feature matrix for the classifiers in Task 2.

Note: I used only the training split (1,049 samples) for all EDA computations to avoid leaking test information into the feature engineering process. If I had included test and validation data and used those insights to engineer features, I would be indirectly leaking information that Task 2 models could exploit. The training split alone was large enough to give statistically sound results.

1.1 Feature 1: Type-Token Ratio (TTR) and MATTR

TTR measures vocabulary diversity as the ratio of unique words to total words. A higher TTR means the author uses more unique words relative to total words. I expected that Victorian authors would score highest here, with AI following it. [1]

While reading about TTR, I came across a paper called "Cutting the Gordian Knot: The Moving-Average Type-Token Ratio (MATTR)" by Covington & McFall (2010). They point out that TTR shrinks as text length increases because "the longer the text goes on, the more likely it is that the next word will be one that has already occurred." MATTR fixes this by sliding a fixed-size window across the text and averaging the per-window TTR values. [2] [3]
I used a window size of 50 words (the standard 100-word window is too large for 150-200 word paragraphs).
My expectation: MATTR of Class 1 > MATTR of Class 3 > MATTR of Class 2

Observations

MATTR results with a window of 50: Class 1 = 0.797, Class 2 = 0.833, Class 3 = 0.812.
I expected Class 1 to have the highest MATTR, but it is actually the lowest. Surprisingly, Class 2 has the highest MATTR, and Class 3 is in the middle as expected. The expectation that "humans have higher TTR than AI" holds at corpus scale, but my paragraphs are just 150-200 words long. At this scale, generic AI shows higher local vocabulary diversity. LMs are trained to avoid local repetition, while Victorian proses rely heavily on repeated dialogue tags ("said", "replied"), character names, and connective phrases ("and then", "and so"). Within a 50-word window, these repetitions bring the MATTR down for Class 1. [2] [4] [5]

Conclusion: MATTR at paragraph scale is a weak feature for Tier A classification, but I included it in the feature matrix anyway.

1.2 Feature 2: Hapax Legomena

A hapax legomenon is a word appearing exactly once in a corpus. A higher count means a more diverse vocabulary. I hypothesised that Victorian authors would have the highest hapax rate, citing Gehrmann et al. (2019) who showed that human-written text contains words a language model considers unlikely significantly more often than AI-generated text. [6] [7] [8]

My first attempt at 5,000 words failed this hypothesis: Generic AI showed the highest hapax rate (22.64%), followed by Mimic AI (20.20%) and Human (19.76%). This is the same reversal I saw in MATTR. At 5,000 words, AI paragraphs generated independently show high diversity because each paragraph is fresh. Human paragraphs share character names and dialogue tags that reduce the hapax rate in small samples.
I then tried sample sizes of 5,000, 20,000, 50,000, and the full corpus. At full corpus scale, the original novel texts had the highest hapax rate (5.29%), confirming the GLTR paper at corpus scale. However, Mimic AI had the lowest hapax rate at larger sizes because the style prompt forced thematic repetition of Gothic vocabulary ("wretched", "desolate", "sublime").
Revised order: Class 1 > Class 2 > Class 3.

Note: I did not add Hapax Legomena to the per-paragraph feature matrix because it is a corpus-level feature, meaningless at paragraph scale. It served as exploratory evidence only.

1.3 Feature 3: POS Distribution

I used SpaCy's POS tagger to extract four metrics per paragraph: adjective-noun ratio, adverb density (per 100 words), past-tense verb ratio, and noun density.

- **Adjective-Noun ratio**: AI models (especially generic ones) usually over-describe. They use more adjectives per noun in an attempt to be more "complete".

- **Adverb density**: The number of adverbs per 100 words. Language models often load text with adverbs like "importantly", "interestingly", "notably", etc.

- **Past tense verbs ratio***:* Shelley's **Frankenstein** and Dickens's **Great Expectations** are both written in the past tense narrative. Generic AI may majorly use present tense since it matches the training data of a lot of language models. Style-mimicking AI, if the prompt is working, should follow Shelley's style and write in past tense.

- **Noun density**: The number of nouns per 100 words. It is a general measure of how "dense" the text is with entities. [9] [10] [11] [12] [13]

Observations:

- **Adjective-Noun Ratio:** Both AI classes scored higher than Human, but Mimic AI scored highest. I expected Generic AI to be highest, but Gemini followed the style prompt and used excessive adjectives to imitate Shelley's Gothic writing. The multi-model composition of Class 3 (45% text by Gemini 2.5 Flash) may have contributed to this.

- **Adverb Density:** Human text had the highest adverb density. This was opposite to my expectation. Victorian prose genuinely used a lot of adverbs for dramatic effect, while modern AI uses common adverbs like "importantly" and "notably" that form only a small subset of all adverbs.
- **Past-Tense Ratio:** Class 1 had a 58% past-verb ratio. Both AI classes were at around 8%. The style prompt did almost nothing for Class 3's tense register. This became the single most discriminative feature of the entire EDA.
- **Noun Density:** Both AI classes had higher noun density than Human. AI seems to name, categorise, and describe things more explicitly, while human writing relies more on pronouns and context.

1.4 Feature 4: Dependency Tree Depth

A dependency parsing tree represents grammatical structure, where each word is linked to a head word through a labelled directed edge. I computed the mean depth of this tree across all sentences in a paragraph as a proxy for syntactic complexity.
My expectation: Depth (Class 1) > Depth (Class 3) > Depth (Class 2). [14] [15] [16] [17]

I used SpaCy's *token.head* and *token.children* methods with a recursive approach to find the sentence root and then count nesting levels. According to SpaCy documentation and a few examples on the internet, simple sentences usually have a depth of around 2-4 levels, whereas complex sentences can have depths of 6-10 levels or more. [18] [19]

Observations:

My hypothesis failed again. Class 3 achieved the highest mean depth. The style prompt asked Gemini for "long complex sentences with nested subordinate clauses", and Gemini took this literally, producing syntactically complex sentences with deep dependency trees. SpaCy's dependency parser is trained on modern English corpora (OntoNotes5, Universal Dependencies treebanks) focused on the late 20th and early 21st centuries.
Victorian English's unfamiliar structure may have forced the parser to produce deep and messier parse trees out of confusion rather than genuine nesting. [20] [21] [22] [23]

The standard deviation observation was also interesting: Human std (1.33) was the highest, confirming that Dickens and Shelley's sentence complexity varied a lot, while AI classes had lower std meaning more uniform sentence structures. I flagged this feature as likely noise for Tier A and dropped it during training (see Task 2).

1.5 Feature 5: Punctuation Density

I was excited for this step because punctuation reflects the author's writing conventions. I computed the frequency of six punctuation marks per 1,000 words: comma, semicolon, colon, em-dash, exclamation, and question mark. [25] [26]

This was the cleanest finding in the entire EDA. There is a 10x jump from Class 2 and an 18x jump from Class 3 to Class 1 in semicolon usage. Em-dash showed a similar pattern. Victorian authors used these marks to create rhythm within long sentences; neither the Generic AI nor the Mimic AI could replicate this pattern. Generic AI returned zeros for exclamation marks, question marks, and parentheses, confirming that a neutrally prompted LLM reflects the training data distribution where commas, periods, and quotation marks dominate.

Comma rate was a surprise. Mimic AI had the highest comma rate (85.90 per 1,000 words). Gemini tried to write complex Victorian sentences by merging clauses with commas instead of semicolons. It achieved some syntactic complexity but used "safe" commas where Shelley would have used semicolons. Wherever Shelley would have placed a semicolon, Gemini placed a comma.

1.6 Feature 6: Readability Indices

The Flesch-Kincaid Grade Level (FK Grade) assesses the approximate reading grade level of a text based on average sentence length and word complexity. The Flesch Reading Ease (FRE) is the inverse: higher FRE means easier to read. I expected Class 1 to have the highest FK Grade, followed by Class 3 and then Class 2. [27] [28]

My hypothesis was partially right. I correctly predicted that Mimic AI would have a higher FK Grade than Generic AI. But I did not expect the Human class to come last. My Class 3 prompt explicitly asked for long complex sentences and formal Victorian vocabulary. Gemini followed this and produced texts with a very high FK Grade.
The wide standard deviation (4.36) for Human highlights the contrast between Dickens's more accessible scenes and Shelley's complex passages. SpaCy's aggressive splitting at semicolons may also have increased the sentence count per paragraph and lowered the average sentence length, directly affecting the FK Grade. [29]

Note: FK Grade turned out to be a useful feature in Tier A but with an inverted correlation to what I assumed —a higher FK Grade meaning more likely AI in my dataset.

1.7 Feature Matrix, PCA, and UMAP

After computing all 15 features (TTR, MATTR, adj-noun ratio, adverb density, past-tense ratio, noun density, tree depth, comma rate, semicolon rate, colon rate, em-dash rate, exclamation rate, question rate, FK Grade, reading ease), I compiled a feature matrix and saved it as *train_features.csv* for use in Task 2.

I then visualised the feature space with PCA and UMAP. PCA compressed 15 features into two principal components capturing **49.4%** of the total variance (PC1: 37.9%, PC2: 11.5%). PC1 separated Human from both AI classes. Every Human paragraph sat on the positive side of PC1 and almost all AI paragraphs on the negative side. [30] [31]

Human characteristics (positive PC1 loadings):

> *reading_ease, past_tense_ratio, semicolon_rate, exclamation_rate, emdash_rate*
> These are the features where Victorian texts differ from AI-generated ones.

AI characteristics (negative PC1 loadings):

> TTR, FK Grade, noun density, tree depth, adj-noun ratio.
> These are the features where my hypotheses reversed, i.e., AI scored higher on all of them. The negative loadings confirm that the reversals were real and consistent, not just noise. Higher FK Grade, higher noun density, and deeper parse trees proved to be AI giveaways in this dataset, even though they are counterintuitive.

UMAP (*n_neighbors*=15, *min_dist*=0.1) preserved local neighbourhood structure and the result was striking. All three classes formed distinct clusters with minimal overlap. Human text was completely isolated while Generic AI and Mimic AI were adjacent but distinguishable clusters. The catch is at the boundary between Generic AI and Mimic AI, where a few AI paragraphs scattered near the Human cluster. These are the paragraphs where Gemini closely mimicked human style. [32]

1.8 Conclusion

After computing 15 stylometric features across 1,050 training paragraphs, I can conclude that the three classes are mathematically distinct. However, the fingerprints are not what I initially expected. The punctuation analysis was the cleanest finding in this entire notebook. Semicolons and em-dashes appeared 10-18x more frequently in human text than in AI text. Past-tense verb ratio was the strongest single discriminative feature (Human: **58%**, both AI classes: around **8%**). I also prepared the 15-dimensional feature matrix for Task 2.

Task 1 — AI and Tool Usage

The analytical framing for PCA and UMAP visualisation (what to expect and how to interpret) was inspired by DeepSeek's Reasoning LLM and supported by my own study of IBM's article, CMU's lecture PDF, and the UMAP documentation. I used the LLM as a discussion partner to test my reasoning before writing it into the notebook. The hypotheses and observations in the MD cells are my own conclusions after looking at the outputs, informed by that discussion.

For code, GitHub Copilot assisted with the SpaCy pipeline and matplotlib visualisation cells. I wrote the feature extraction functions (MATTR, punctuation density, FK Grade, etc.) myself based on the approaches described in the cited sources. I also used Bohrium AI to locate the relevant sections of Gehrmann et al. (2019) without reading the full paper.

Task 1 References (hyperlinked)

1. Glossary: Type-Token Ratio (TTR). Sketch Engine.

2. Human vs. LLM Creativity: A Comparative Analysis of Task-Dependent Asymmetry and Linguistic Features. PMC.

3. Covington, M. A., & McFall, J. D. (2010). Cutting the Gordian Knot: The Moving-Average Type–Token Ratio (MATTR). Journal of Quantitative Linguistics, 17(2), 94–100.

4. AI Content Detection vs Human Review: What Really Works in 2025? Skyline Academic.

5. Large Language Models (LLMs) Course. Dr. Kshitiz Verma.

6. "Adjusting to Victorian Writing Style & Language". Goodreads.

7. Sebastian Gehrmann, Hendrik Strobelt, and Alexander Rush. 2019. GLTR: Statistical Detection and Visualization of Generated Text.

8. A first excercise in natural language processing with Python: counting hapaxes. Catswhisker.

9. Lexical Density. Vocabulary Matters.

10. Section 2.1 of "PUCP Matrix: An Open-source and Comprehensive Toolkit for Linguistic Analysis of Spanish Texts". EACL 2026.

11. Text Analysis without AI: Lexical Density, String Similarity, Readability, Other Metrics. Greenflux Blog.

12. Chapter 5: Categorizing and Tagging Words. NLTK Book.

13. NLP Part of Speech Default Tagging. GeeksforGeeks.

14. "19. Dependency Parsing". Stanford NLP Course.

15. "Dependency Parsing in NLP". ProjectPro.

16. "Dependency Parsing: Associated Algorithms in NLP". Medium.

17. "Dependency Parsing in Natural Language Processing with Examples". Analytics Vidhya.

18. "NLP Sentence Dissection by Dependency Parsing with SpaCy". LevelUp.

19. "Linguistic Features". SpaCy Documentation.

20. SpaCy Dependency Parser Discussion. GitHub.

21. StackOverflow: Why does SpaCy's dependency parser produce deep trees for simple sentences?

22. SpaCy API: DependencyParser.

23. "SpaCy's DependencyParser: What Goes on Behind the Scenes". Medium.

24. "The Corpus of Late Modern English Texts (CLMET)". ICAME.

25. "NLP Punctuation, Lower Case, and Stopwords Pre-processing". Medium.

26. "From Words to Numbers: A Deep Dive into NLP Feature Engineering". LevelUp.

27. "Flesch Reading Ease & Flesch-Kincaid Grade Level". Readable.

28. "Flesch-Kincaid Grade Level: A Key Tool for English Educators". Text Inspector.

29. Linguistic Features. SpaCy Documentation.

30. "Principal Component Analysis". IBM.

31. "Lecture 18: Principal Component Analysis". CMU Statistics.

32. "UMAP Plotting". UMAP Learn Documentation.

33. DeepSeek.

34. Bohrium AI.

Task 2: The Multi-Tiered Detective

Task 1 established that my stylometric features separate the classes. Task 2 asks the machine to actually use those features to classify. I built three classifiers of increasing sophistication:

- **Tier A (The Statistician):** XGBoost on the 15 stylometric features from Task 1.
- **Tier B (The Semanticist):** Feedforward Neural Network on averaged GloVe word embeddings (300-dimensional).
- **Tier C (The Transformer):** Fine-tuned DistilBERT with LoRA.

My hypotheses going into Task 2:

- **H1:** Tier A will easily separate Human from Generic AI (past_tense_ratio and semicolon_rate are strong), but will struggle with Human vs Mimic AI.
- **H2:** Tier B will perform the worst. All three classes cover the same 10 topics, so averaged GloVe embeddings encoding semantics will not distinguish classes that differ in writing style.
- **H3:** Tier C will perform best overall, since a contextual transformer sees syntax, rhythm, and token distribution patterns.
- **H4:** past_tense_ratio and semicolon_rate will be Tier A's top features by gain. Task 1 showed these have the largest class separation.
- **H5:** Mimic AI will be the most confused class because it is prompted to sit between Human and Generic AI.

2.1 Tier A - XGBoost

I chose XGBoost over Random Forest because its gain-based feature importance is more informative. Random Forest averages impurity across trees in a way that can mask redundant features, while XGBoost's gain directly tells which features contribute to splits that actually reduce error. I also wanted hands-on practice with it. [1] [2] [3] [4]

I tuned three hyperparameters using a 5-fold stratified Grid Search on the training set: *n_estimators* [100, 200, 500], *max_depth* [3, 5, 7], and *learning_rate* [0.01, 0.1, 0.3].

3 values x 3 values x 3 values = **27 combinations** (if tried by brute-force).

After hyperparameter tuning: *max_depth* = 3, *n_estimators* = 500, *learning_rate* = 0.3.
Shallow trees (depth=3) imply that the actual decision boundary was simple since a single feature (*past_tense_ratio*) nearly solved the Human vs. AI boundary alone.
Many trees at high learning rate provided ensemble stability without overfitting, probably

because each tree is too shallow to memorise the training samples. [5] [6] [7]

Tier A Results: 98.22% test accuracy, macro-F1 = 0.98

This is too good to be true for a 3-class NLP problem where I engineered the features myself. If I consider these results accurate, the stylometric signal is strong enough that a simple XGBoost already solved the task.

Hypothesis evaluation:

- H1 rejected. I predicted Tier A would struggle with Human vs Mimic AI, but the confusion matrix shows zero errors in both directions. past_tense_ratio (Human=0.58, Mimic AI=0.09) made this an almost trivial split. Even though Gemini was prompted to write like Shelley, it used present tense anyway.
- H4 partially confirmed. semicolon_rate and past_tense_ratio are both in the top 3 features, matching my prediction. But comma_rate is the most important, not past_tense_ratio. After a discussion with DeepSeek's reasoning LLM, I think this is because comma_rate creates a 3-way split: Mimic AI (85.90) > Human (75.40) > Generic AI (48.85), while past_tense_ratio only helps with a 2-way split (Human vs both AI classes). Features that separate all three classes simultaneously get higher gain. [8]
- H5 rejected. Mimic AI was the easiest class (100% recall on both val and test). Generic AI was the most confused class with 3 total errors.

I also ran a reduced feature set experiment by dropping *tree_depth* and *adv_per_100* (both showed inverted correlations and heavy class overlap in Task 1). The model still achieved 98.22% overall accuracy but improved Mimic AI F1 from 0.99 to 1.00, confirming that these two features were adding noise.

2.2 Tier B - Feedforward NN

GloVe (300d) is a pre-trained lookup table that maps each word to a 300-dimensional vector, placing words with similar context nearby in the vector space. I averaged the GloVe vectors of all words in a paragraph to get a single 300d representation. Averaged GloVe vectors encode what a paragraph is about, i.e., semantic content rather than writing style. Since all three classes cover the same 10 topics, I expected near-chance performance. [1] [2] [3] [4]

Network architecture:

- Two hidden layers (256, 128) with ReLU activations and dropout (p = 0.3).
- Adam optimiser with lr = 1e-3. Early stopping with patience=7.
- Max 50 epochs. OOV (out-of-vocabulary) words skipped; OOV rate was low across all splits. [5] [6] [7] [8] [9]

Tier B Results: 100% test accuracy and macro-F1 = 1.0 (A nightmare)

My first thought after seeing the results was that data had leaked. I triple-checked every piece of code that processes the data and found nothing. I then consulted DeepSeek's reasoning LLM. It suggested that maybe GloVe embeddings encode style in a way I did not expect. [10]

After going through my previous observations and the cited articles, I revised my hypothesis: I was thinking of semantics as "what the paragraph is about". The gap is that GloVe embeddings also encode which specific words were used. Victorian proses (Class 1) use "wretched", "melancholy", "sublime", etc.
AI-generated texts use "navigate", "framework", "dynamics", etc. These word sets live in completely different regions of the 300d feature space. Averaging Class 1 word vectors lands in the "Victorian neighbourhood", while averaging AI classes lands in the "modern English neighbourhood."
Even though the topics were the same, the vocabularies were completely different. The per-class OOV analysis also confirmed this: Human text had a 0.7% OOV rate vs. 0.0% for Generic AI and 0.1% for Mimic AI, meaning Victorian character names and rare vocabulary do not appear in GloVe's training corpus. [11]

The training curve confirmed this: train and val loss tracked each other from epoch 1 with little gap, both converging to near-zero by epoch 7-8. The model found a really easy, generalizable pattern. No overfitting.

Revised H2: Averaged GloVe embeddings did not fail on this task because lexical choice and writing style came together. The vocabulary choice is the style signal in this dataset. However, the limitation of Tier B is still real: the 100% result is due to the gap between Victorian proses and modern English LLM outputs, not genuine semantic discrimination.

2.3 Tier C - DistilBERT & LoRA

DistilBERT (Sanh et al., 2019) is a **40% smaller**, **60% faster** distilled version of BERT-base that retains around **97%** of BERT's performance. I chose it over RoBERTa because it is smaller and easier to train, and my paragraphs (150-200 words) fit well within the 512-token limit. [1]

I used LoRA (Hu et al., 2021) as instructed. The idea is that the changes needed to adapt a pre-trained model to a specific task can be captured by a small number of parameters. Instead of updating all 66M parameters, LoRA inserts small trainable rank-decomposition matrices into the attention layers: the effective weight becomes W + (A x B), where A has shape *(768 x r)* and B has shape *(r x 768),* with **rank r = 8**. This reduces parameters from *768x768 = 589,824* to *768x8 + 8x768 = 12,288* per matrix. A 48x reduction! [2]

LoRA Configuration:

- **rank (r) = 8**, **lora_alpha = 16**, **lora_dropout = 0.1**
- Target modules: **q_lin** and **v_lin** (Query and Value projection matrices). The original LoRA paper found that adapting Q and V provides most of the fine-tuning benefit. Intuitively, adapting Q and V is enough to direct what the model pays attention to and what it extracts, without re-learning the matching criterion from scratch.
- **lora_alpha = 16 = 2 * r** (the LoRA authors' standard default). The actual update applied is *(lora_alpha / r) * A x B*. Too low means the optimizer might ignore the update; too high risks catastrophic forgetting of pre-trained knowledge. [2] [6]
- **Learning rate: 2e-5** (community-standard for BERT-family fine-tuning, validated across hundreds of downstream tasks). **warmup_steps = 100** to avoid a sharp gradient spike at the start of training and protect pre-trained weights. [8]
- **weight_decay = 0.01** (L2 regularization). Early stopping **patience = 2** on *val macro-F1*.

The model print showed **740,355** trainable parameters, which is **1.09%** of total.
This is in line with the expected (*12,288* parameters per matrix) x (2 matrices) x (6 layers) = **147,456 LoRA parameters**, **plus 592,899** for the **new classification head** that was randomly initialised since DistilBERT was pre-trained for masked language modeling, not sequence classification.

Tier C Results: 100% test accuracy and macro-F1 = 1.0

Such perfect results for this dataset were no longer a surprise after Tier A and B. Early stopping triggered at epoch 4 because val macro-F1 plateaued at 1.0 from epoch 2 onwards.

This is not overfitting. Val loss decreased every epoch (1.023 -> 0.711 -> 0.354 -> 0.197). Overfitting would have shown val loss increasing while train loss decreased continuously. The model found a clean decision boundary by epoch 2 and the remaining epochs refined internal representations without changing the predictions.

H3 is confirmed in the sense that Tier C performed at least as well as Tiers A and B. It did not outperform them because there was no room. All three tiers hit ceiling performance. The more meaningful question is what each tier detected. Tier A detected stylometric signatures (past tense, semicolons, commas). Tier B detected lexical choice (Victorian vs modern AI vocabulary). Tier C, a transformer, read subword patterns, attention distributions, and contextual relationships not expressible as just 15 numbers.

LoRA only updated *1.09%* of the parameters, yet it was enough to achieve **100% accuracy** on a domain shift from general English to Victorian proses. DistilBERT's pre-trained representations already contained features relevant to this task and LoRA just redirected them to my classification boundary.

Overall Conclusion of Task 2

Three very different architectures: a gradient-boosted tree, a feedforward neural network on averaged embeddings, and a fine-tuned transformer. All of them reached near-perfect accuracy. This means the stylometric gap between human-written Victorian proses and AI-generated texts is not subtle. Any model that looks carefully at language patterns will find it.

Note: I began this task intending to "catch the ghost", but it turns out my ghost was just standing with a flashing neon sign.

Task 2 - Tier A References (hyperlinked)

DeepSeek's Reasoning LLM (cited in-line) helped me interpret the feature importance output and understand how *comma_rate* created a 3-way split. I structured hyperparameter tuning using *GridSearchCV* based on the concepts explained in my cited articles. GitHub Copilot in my IDE assisted with the code structure for the XGBoost training loop while I came up with the logic and adapted its code suggestions to my features.

1. XGBoost Feature Importance. Medium (Emily Marsh).
2. XGBoost Readthedocs. XGB Importance.
3. Gini Impurity. Medium (Hisah).
4. Feature Importance Gain in XGBoost. StackOverflow.
5. Optimizing XGBoost: A Guide to Hyperparameter Tuning. Medium (Dicee).
6. GridSearchCV Concepts. Medium (Mandeep).
7. Tune Hyperparameters with GridSearchCV. Analytics Vidhya.
8. DeepSeek conversation on feature importance interpretation and overall observations assistance.
9. GitHub Copilot Assistant in VS Code.

Task 2 - Tier B References (hyperlinked)

For boilerplate code and general understanding of GloVe, I referred to the original paper and the official Stanford NLP project page. Bohrium AI helped summarize the GLoVe architecture and pointed out important sections for me to skim through because it was not feasible to read the entire paper in detail given the time constraints.

As cited in-line, I used DeepSeek for understanding what a 100% accuracy and 1.0 macro-F1 score could mean. It gave me common interpretations and suggestions based on its training

data because it didn't have my entire notebook context. I connected the dots and came up with the revised hypothesis on my own based on DeepSeek's insights and my previously cited resources. [12]

For coding assistance, I used Github Copilot Assistant in my VS Code. It was handy for the Dataset and FFNet classes' code snippets and structural suggestions. However, since my Copilot's knowledge cutoff is October 2024, it often failed to provide accurate parameter suggestions for the latest versions of PyTorch, Sklearn, and XGBoost. I had to cross-reference with the official documentation and the other cited sources.

1. GloVe: Global Vectors for Word Representation. Pennington et al. EMNLP 2014.
2. GloVe Embeddings. Stanford NLP.
3. Mastering NLP with GloVe Embeddings: Word Similarity, Sentiment Analysis, and More. Medium (Muneeb).
4. GloVe Glossary. Zilliz.
5. Deal with Out-of-Vocabulary Word with Gensim Pretrained GloVe. StackOverflow.
6. Deep Learning Basics Part 10: Feed Forward Neural Networks (FFNN). Medium (Sasirekhar A).
7. Understanding Dropout in Deep Neural Networks. Medium (Codex).
8. Demystifying the Adam Optimizer in Machine Learning. Medium (Weidagang).
9. Regularization for Neural Network. Halim Noor.
10. DeepSeek conversation on Tier B results interpretation and revised hypothesis formulation.
11. Discussion on GloVe embeddings encoding style. Kaggle.
12. Bohrium AI – GLoVe Architecture Summary.
13. GitHub Copilot Assistant in VS Code.

Task 2 - Tier C References (hyperlinked)

To go through the most-relevant sections of the DistilBERT and LoRA papers, I used Bohrium AI. It helped me figure out which portions of the papers to read in detail and which ones to skim. Reading the paper helped me with my LoRA config choices (rank = 8, lora_alpha = 16, targeting Q and V matrices), which I cross-verified with HuggingFace PEFT documentation and previous works documented as Medium articles (cited). [10]

I used Github Copilot in my VS Code IDE for coding assistance while implementing the fine-tuning. It was helpful for forming a general structure, which I had to verify from previous works on Medium and HuggingFace (cited). However, Copilot constantly gave outdated suggestions

for the latest versions of some libraries. For example, it suggested `tokenizing_class` instead of `processing_class` in the HuggingFace Trainer, which is a recent change. I had to go through the documentation to figure out the correct parameters. I also used it to generate the training loop structure, and verified it against the HuggingFace Trainer documentation and published notebooks (cited). [11]

1. DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. ArXiv (Sanh et al., 2019).

2. LoRA: Low-Rank Adaptation of Large Language Models. ArXiv (Hu et al., 2021).

3. Parameter Efficient Fine-Tuning using LoRA. Medium (Techsalo Infotech).

4. Fine-tuning BERT for Text Classification with LoRA. Medium (Karkar Nizar).

5. A Study to Evaluate the Impact of LoRA Fine-tuning on the Performance of Non-functional Requirements Classification. Arxiv (Li et. al., 2025).

6. An Introduction to LoRA: Unpacking the Theory and Practical Implementation. Medium (Mujahid Abdullahi).

7. Fine-tuning Large Language Models (LLMs). Medium (Data Science).

8. Catastrophic Forgetting in LLMs. Medium (Cobus Greyling).

9. HuggingFace PEFT Checkpointing.

10. Bohrium AI.

11. Github Copilot assistant in VS Code. Github.

Task 3: The Smoking Gun

Task 2 did not end the way I expected, but I think I interpreted the results fairly well.

Task 3 asked me to find three samples where Human was labelled as AI. There is an obvious problem: there are no such samples because the model never got it wrong on the test set. This is a good problem to have on paper, but I cannot do the error analysis as originally planned. So I adapted the methodology to still get insights about how the model makes its decisions:

- Saliency analysis using Integrated Gradients (Captum) on correctly classified samples to show which tokens the model relied on for each class.
- Near-miss analysis: since there are no misclassifications, I found the correctly classified samples where the model was least confident. These are the most human-like AI paragraphs and the most AI-like human paragraphs.
- Adversarial probing: constructing targeted paragraphs to challenge the model and watch where its confidence drops.

Integrated Gradients was introduced in "Axiomatic Attribution for Deep Networks" (Sundararajan et al., 2017). The idea is to see how much each token caused the model's prediction. Starting from a baseline input (all padding tokens — zero information), the algorithm interpolates between the baseline and the actual input, computes gradients at intermediate points, and integrates them to get attribution scores per token. A high positive score means the token pushed the model towards the predicted class; a high negative score means it pushed away. The completeness axiom ensures scores sum to the total difference in model output between actual input and baseline. I verified this via convergence deltas (all close to zero, meaning reliable attributions). [1] [2] [3] [4] [5] [6]

My hypotheses:

- **H1:** The model will detect specific vocabulary words distinguishing Victorian prose from modern AI text, since Tier B (GloVe FFNN) already achieved 100% accuracy on word-level signals alone.
- **H2:** Past-tense verb forms will receive the highest attribution scores for the Human class, given that past_tense_ratio was the most discriminative stylometric feature in Task 1.

3.1 Saliency Analysis

I ran Captum's IntegratedGradients on one representative sample from each class. The attribution plots coloured tokens blue (positive attribution —pushes towards predicted class) and red (negative attribution — pushes away).

Observations:

- Human class: The highest attributions were on narrative and emotional first-person words ("emotions", "humanity", "impressions", "lost"). The word "I" appearing repeatedly with high attribution reinforced my belief that first-person past-tense is the model's primary Human signal.
- Generic AI: The top attributed tokens were almost entirely punctuation (commas and periods) and function words like "the" and "is". Almost no content words appeared. The model has probably learned that Generic AI uses almost no expressive punctuation, and is flagging the presence of commas and periods as the only available punctuation signal.
- Mimic AI: Gothic vocabulary like "suffering" and "despair" dominated the attributions. The model learned that this vocabulary belongs to AI trying to mimic authentic Victorian prose. Counterintuitively, "despair" is a Mimic AI signal, not a Human signal. Real Victorian authors use this vocabulary in complex past-tense sentences, while Mimic AI uses it in present tense.

H1 partially confirmed. The model detects both lexical signals (Gothic vocabulary for Mimic AI) and structural signals (punctuation patterns for Generic AI).

H2 confirmed. Past-tense verb forms and first-person "I" have the highest attribution scores for the Human class.

3.2 AI-isms Investigation

One sample per class is not enough for general conclusions. I ran attribution across 5 samples from each class, aggregated the top-10 positively attributed tokens per sample, and counted how often each appeared.

- Human class: Character names (e.g., "Joe" from Great Expectations, appearing 3 times), emotional content words ("emotions", "humanity"), and expressive punctuation (exclamation marks, curly quotation marks). The model learned that Victorian writing has named characters, emotional vocabulary, and a variety of punctuation marks.
- Generic AI: Almost only punctuation (13 commas, 10 periods) and function words ("the", "is"). Almost no content words. Generic AI is identified by what it lacks (semicolons, em-dashes, exclamation marks).
- Mimic AI: 16 commas, 8 periods, and Gothic vocabulary like "suffering", "despair", "winter". Comma dominance confirmed the Task 1 observation that Mimic AI substituted commas for semicolons. But Gothic words also served as important AI-isms for this class.

The model does not rely on a single mechanism. Human classification uses content words and expressive punctuation. Generic AI is detected by absence of complex punctuation. Mimic AI is detected by Gothic vocabulary and comma density. Three different signals for three different problems — which is why all three tiers in Task 2 achieved 100% accuracy.

3.3 Near-Miss Analysis

Since no samples were misclassified, I found the correctly classified samples where the model was least confident. These paragraphs lived closest to the decision boundary.

Human near-misses (confidences 0.424-0.433):

The three least-confident Human samples all had Mimic AI as the second prediction (0.40-0.41). The model was confusing real Shelley with the AI-mimicked Shelley, not with modern generic text. The attribution plot of near-miss Human sample 3 showed Gothic vocabulary like "archangel", "devil", and "veins" in bright red (highly negatively attributed). These tokens were pushing the model away from its Human prediction.
When real Shelley used her most emotionally extreme passages, she accidentally triggered the same signal the model associates with Mimic AI.

AI near-misses (confidences 0.372-0.387):

All three least-confident AI samples came from the same topic: "Crime, guilt, and redemption." This is not a coincidence. This topic naturally induces third-person past-tense narration ("he walked", "he realized"). When Generic AI accidentally narrated a story about crime and guilt, it mimicked the most human-like syntactic feature in the dataset, and the model got confused. The second near-miss AI sample showed a paragraph with past-tense narration ("he returned to the alleyway", "he stood before the boarded store"). The word "soul" also appeared with negative attribution — Gothic vocabulary in generic AI prose pulled the prediction towards Human.

After all of this, I can say that the decision boundary is not where I would have drawn it at the end of Task 1. The model did not learn human writing style in a deep sense. It learned surface-level patterns (tense and vocabulary distribution) that happen to correlate with class descriptions in this dataset. At the boundary, those patterns were not strong enough and the model's confidence crashed.

3.4 Adversarial Probe

The Near-Miss analysis showed where the decision boundary lies. I wanted to test whether I could cross it deliberately. The idea came from skimming the OUTFOX paper (Koike et al., 2023), where the authors proposed a feedback-loop adversarial attack. My version is simpler but follows the same principle. I know exactly which features the model uses (from Integrated Gradients), so I do not need to shoot in the dark. [7]

I prompted DeepSeek-V3 to generate two paragraphs: one with modern AI-like vocabulary written entirely in past tense, and one with Victorian vocabulary written entirely in present tense. The prompts were carefully crafted based on the saliency analysis. [8]

Results:

- Test 1 (Modern vocabulary, past tense): Predicted as Generic AI (confidence 0.468). Contrary to my expectation, modern AI-like prose in past tense did not fool the model. The vocabulary ("framework", "dynamics", "navigate", "compliance") was flagged in blue as Generic AI signals. Past tense is necessary for Human classification but not sufficient; it also needs the right lexical style.
- Test 2 (Victorian vocabulary, present tense): Predicted as Mimic AI (confidence 0.471). Gothic vocabulary like "isolation", "dark", "heart" were positively attributed, but without past tense, the paragraph ended up in the Mimic AI region. My Class 3 generation prompt asked Gemini for Victorian vocabulary in present tense, and the model actually learned that combination as the Mimic AI signature.
- Test 3 (Real Shelley text, ground truth): Correctly classified as Human, as expected for a sanity check.

The detector is not naive in the way I expected. Changing one surface feature (tense alone or vocabulary alone) does not flip the classification. The decision boundary requires multiple features to align simultaneously. This explains why even the model's least-confident correct predictions still landed in the correct class, just with lower confidence. Task 4 tests if a Genetic Algorithm can evolve across all features simultaneously rather than attacking them one at a time.

Task 3 - AI and Tool Usage

GitHub Copilot came in clutch with the Saliency Analysis code structure and syntax, as I was jumping between Captum's documentation, tutorials, and online walkthroughs. I had never implemented Integrated Gradients before, and this was a good starting point. The AI-isms Investigation and Near-Miss analysis sections were more custom and I wrote them myself, but Copilot's suggestions were accurate on structure. Bohrium AI helped me identify the exact sections of the OUTFOX paper relevant to my scope. I discovered the paper through a Kaggle discussion and went through the sections Bohrium suggested. DeepSeek-V3 was used to generate the two adversarial test paragraphs.

Task 3 References (hyperlinked)

1. Sundararajan, M., Taly, A., & Yan, Q. (2017). Axiomatic attribution for deep networks. In International Conference on Machine Learning (pp. 3319-3328). PMLR.

2. Deep Dive into Neural Network Explanations with Integrated Gradients. Towards Data Science.

3. Kaggle Discussion on Integrated Gradients.

4. Captum Integrated Gradients API.

5. Using Captum for Interpreting PyTorch Models in Production. Medium.

6. Captum Tutorial on BERT SQuAD Interpretation.

7. OUTFOX: A Benchmark for Evaluating and Enhancing the Robustness of LLM Detectors. Koike et al., 2023.

8. Chat with DeepSeek-V3 for Adversarial Test texts generation.

9. Research assistance with Bohrium AI.

Task 4 — The Turing Test

I structured Task 4 into two parts: Part 1 uses a Genetic Algorithm to evolve AI-generated text through guided mutations to maximise the Tier C model's P(Human) score. Part 2 runs a personal essay through the same detector and compares it against an AI-style rewrite of the same content.

My hypotheses:

- **H1:** The GA will improve P(Human) over generations but will probably not reach 90%. Task 3's adversarial tests showed that single-feature mutations did not flip the classification. Multiple features (past tense, Victorian vocabulary, syntactic complexity) need to come together, and the GA must discover this combination organically.
- **H2:** My essay will be classified as Generic AI or borderline, not fully Human. In the past three tasks I have seen that the model has no reference for how I write. Its Human class was trained on Victorian fiction, so any real modern writing will fall outside the decision boundary.

4.1 Part 1: Genetic Algorithm

4.1.1 Initial Population

I generated the initial population by calling the Gemini API on the topic "Crime, guilt, and redemption". I identified this as the topic that produced the highest baseline P(Human) scores in Generic AI text in Task 3's Near-Miss analysis, because it naturally induced past-tense narrative (Near-Miss paragraphs on this topic had P(Human) in the range 0.263-0.309).
I used a neutral prompt (no style instruction) so the GA had room to evolve style through mutations, with temperature 0.9 for initial diversity. The model used was *gemini-2.5-flash-lite*, managed through the same KeyRotator class from Task 0.

The mean P(Human) of the initial population was **0.2994**: very near the random baseline of 0.333 for a three-class classifier. The best starting P(Human) was **0.3400**, barely above random. Interestingly, six out of ten paragraphs were classified as Mimic AI. The *"crime, guilt, and redemption"* topic probably pushed Gemini into dark and emotional prose, which the model confused with attempted Victorian mimicry.

4.1.2 GA Loop Design

The GA ran for up to 7 generations with population size 10 and 3 survivors per generation (*TOP_K* = 3). Each generation kept the top 3 survivors by P(Human), then generated 7 children through mutations. Early stopping triggered if any individual reached P(Human) >= 0.90.

Three mutation strategies were designed from Task 3's findings:

- Mutation A (Tense register): Convert all verbs to past tense and add first-person narrative observations ("I noticed", "I felt", "I understood"). It targets *past_tense_ratio* — the strongest discriminative feature from Task 1.
- Mutation B (Vocabulary): Replace modern corporate-sounding words ("navigate", "framework", "dynamics", "context", "trajectory", "compliance") with older, more concrete English words. Add one semicolon. Task 3 showed that Victorian vocabulary was necessary but not sufficient alone.
- Mutation C (Syntax): Add at least two nested subordinate clauses ("which", "who", "although", "whilst") and introduce one em-dash for emphasis, while maintaining past tense throughout. Targets syntactic complexity and em-dash usage identified as collectively necessary in Task 3.

Mutation temperature was **0.7** (more focused output). A 5-second sleep between children kept the per-key call rate under 10 RPM. The KeyRotator handled any 429/quota errors automatically, rotating to the next key and sleeping 30 seconds before retrying.

4.1.3 GA Results

The GA started from a best P(Human) of **0.34**, reached **0.4945** by generation 5, and then completely flatlined. Generations 5, 6, and 7 had the same best paragraph. The algorithm found a local optimum with no path forward.

Hypothesis evaluation:

- **H1 confirmed.** Task 3's adversarial tests correctly predicted that single-feature mutations cannot push a paragraph across the decision boundary because the model requires a combination of features simultaneously. The plateau at around *0.49* confirms this.
- The GA converged at generation 5 because Gemini's small vocabulary and tense changes did not change the overall register. Both best and mean P(Human) rose from generations 0-5, then the gap between them decreased and both flatlined.
- The best paragraph after 7 generations was predicted as Human with **P(Human)=0.4945**, barely above random baseline (0.333). The model's confidence was near-uniform across all three classes. This means the GA got close to the decision boundary but Gemini's definition of "Victorian style" is different from Mary Shelley's Victorian style.

Note: This outcome is not a failure. It is a direct confirmation of Task 3's finding: the Tier C model requires simultaneous alignment of multiple Victorian features, which single-paragraph mutations delivered via Gemini cannot achieve.

4.2 Part 2: Personal Essay Test

I wrote a short personal essay for this test. My friends often joke that I write everything very precisely and in a conversational tone. I was confident the model would classify my writing as AI. The essay covers: first-person narrative, mixed tense (past for events, present for reflection), conversational modern English, no Victorian vocabulary.

I also wrote an explicit AI-style rewrite of the same first paragraph (corporate register, passive constructions, abstract nouns) as a comparison.

Results:

- Essay: Classified as **Generic AI** (P(GenAI) = **0.387**, P(Human) = **0.331**). The essay was close to the three-way boundary, but landed in Generic AI.
- AI-style rewrite: Also classified as **Generic AI**, but with higher confidence (P(GenAI) = **0.482**). The model correctly detected that the rewrite was more AI-like and moved in the right direction.

H2 confirmed. It is not surprising that my essay was classified as AI. My essay has no Victorian vocabulary, no semicolons, no em-dashes, no Gothic prose. None of the features the model associates with Human class. The bar chart also shows that the AI rewrite has a much taller Generic AI bar and a much shorter Human bar compared to the genuine essay, confirming the model moved in the right direction between them.

The task asked whether I could manually lower my "AI score" if the model flags my writing as AI. My answer is that I can, but only by writing like Shelley: adding semicolons, switching to pure past tense, and using complex Victorian vocabulary. But this is not "humanising". This is "Victorianising".

Task 4 - AI and Tool Usage

I designed the three mutation prompts (tense register, vocabulary replacement, syntactic complexity) based on the findings of Task 3.
GitHub Copilot in my IDE generated the Genetic Algorithm code structure after I described my approach. It also helped in writing the mutation functions and the fitness trajectory visualisation code.

Conclusion

The assignment asked two questions: Can I evolve a paragraph that bypasses my detector? Can I fool my machine with my own writing?

The answer to the first question is "almost". Seven generations of evolution pushed a Generic AI paragraph to P(Human) = *0.4945*, which was classified as **Human**. But the GA converged at *0.49* and could not climb further. Thus, I cannot bypass my own detector with simple mutations.
The answer to the second question is "no". Not because I write like AI, but because the model did not learn how humans actually write. It only learned one specific style of human writing. My essay would look "human" to a human reader, but the model sees it as "Generic AI".

I spent thirteen days trying to find the ghost in Victorian texts. My model found it. It can differentiate between Mary Shelley and Gemini with **100% accuracy**. But it cannot recognise a human who does not write like Shelley.

Final Note

I think a better experiment would have been to train the detector on modern human writing along with Victorian prose.
I can also use a Genetic Algorithm with mutations targeting all five Human features simultaneously rather than one at a time. The results will be interesting.

The domain-specificity of my detector is obviously not a failure within its training distribution. But it is a failure of generalisation, which is the central unsolved problem in AI text detection literature.